プログラマ就活ポートフォリオ

C++ / Siv3D / Tiled を用いた2Dアクションゲーム

氏名: 村松 寛太

開発期間: 5週間 (= ゲームジャム:2週間 + ブラッシュアップ:3週間)
環境: Windows 11 / Visual Studio 2022 / Siv3D 0.6.16

目次

1. **はじめに**: 本資料と作品の概要
2. **アピールポイント (i)**: 拡張性の高い敵の行動設計(Strategyパターン)
3. **アピールポイント (ii)**: データ駆動型のレベルデザイン
4. **アピールポイント (iii)**: 可読性とコーディング規約
5. **プログラムの課題**
6. **まとめ**

1. 本資料と作品の概要

本資料について

本資料は，私と友人で共同作成したゲーム作品「シンカイサンタ」の**プログラム**について，**アピールポイントを解説**することを目的としています．

作品概要

深海を探索するというテーマで作成された縦型2Dアクションゲームです．
2週間のゲームジャム期間の中で**基本的なゲームプレイ**を実装し，その後約3週間かけて**設計の改善と品質向上**を図りました．

2. アピールポイント (i): 拡張性の高いAI設計

敵キャラクター(Enemy)の行動ロジックを直接クラス内に記述せず，
Strategyパターンを用いて分離しました.
これにより，新しい敵の行動を追加する際に既存コードへの影響を
最小限に抑えられます.

2.1 インターフェース定義 (IBehaviorStrategy.h)

全ての行動戦略の基底となるインターフェースを定義し，Updateメソッドを通じたポリモーフィズムを実現しています．

```
/// @brief 敵の行動戦略を定義するインターフェース
class IBehaviorStrategy
{
public:
    virtual ~IBehaviorStrategy() = default;

    /// @brief 各戦略ごとの更新処理
    /// @param enemy 対象の敵オブジェクト
    /// @param stage ステージ情報
    virtual void Update(Enemy& enemy, const Stage& stage) = 0;
};
```

2.2 具象クラスの実装 (BehaviorStrategies.h)

具体的な行動ロジック(巡回，待機など)は，このインターフェースを実装する形で定義しています．

```
/// @brief 左右に巡回する行動
class PatrolBehavior final : public IBehaviorStrategy
{
public:
    void Update(Enemy& enemy, const Stage& stage) override;
};

/// @brief その場から動かない行動
class StationaryBehavior final : public IBehaviorStrategy
{
    // ...
};
```

2.3 Strategyの注入 (Enemy.cpp)

敵の生成時，パラメータ(JSONデータ)に応じて適切なStrategyを動的に生成・注入(Dependency Injection)します.

```
// Strategy の生成
switch(spec.behavior)
{
case EnemyDataSystem::BehaviorKind::Patrol:
    behavior_strategy_ = std::make_unique<PatrolBehavior>();
    break;
case EnemyDataSystem::BehaviorKind::BackAndForth:
    behavior_strategy_ = std::make_unique<BackAndForthBehavior>();
    break;
default:
    behavior_strategy_ = std::make_unique<StationaryBehavior>();
    break;
}
```

3. アピールポイント (ii): データ駆動型のレベルデザイン

レベルエディタ **Tiled** のJSONデータを直接読み込み，プログラムのリコンパイルなしでステージ構成や敵の配置を調整可能にしました.
エンジニア以外のメンバー（プランナー等）との協業効率を高める設計です.

3.1 ステージデータのパース (Stage.cpp)

レイヤー名に基づいて，「地形（タイル）」と「オブジェクト（敵・スポーン位置）」を振り分けて処理します．

```
void Stage::LoadFromJson(const FilePath& json_path)
{
    // ...
    for(const auto& layer : json[U"layers"].arrayView())
    {
        const String type = layer[U"type"].getString();

        // 地形データの解析へ
        if(type == U"tilelayer") ParseTileLayer(layer);

        // 敵配置データの解析へ
        else if(type == U"objectgroup") ParseObjectLayer(layer);
    }
}
```

3.2 パラメータの一元管理 (EnemyDataSystem.cpp)

敵の移動速度やHPなどのバランス調整用パラメータは，コード内埋め込み（マジックナンバー）を避け，外部JSONファイルに集約しています．

```
// JSONからパラメータを読み込む
spec.speed = GetDoubleOr(src, U"speed", 0.0);
spec.collision_offset = GetDoubleOr(src, U"collision_offset", 0.0);

// アニメーション定義もJSONから動的ロード
if(src.hasElement(U"animations") && src[U"animations"].isObject())
{
    // ...
}
```

4. アピールポイント (iii): 可読性とコーディング規約

チーム開発でのメンテナンス性を考慮し，一貫した命名規則とドキュメンテーションコメントを徹底しました.

- **PascalCase**: クラス名，関数名 (Update, `StartGame`)
- **snake_case + `_`**: Privateメンバ変数 (`pos_`, `is_alive_`)
- **Doxygenコメント**: 全ての公開ヘッダに `@brief`, `@param` を記述

4.1 コード例 (Player.h)

チーム開発を想定し，一貫性のある命名規則とDoxygenコメントを徹底しています．

```
/// @brief プレイヤーを管理する
class Player
{
public:
    /// @brief コンストラクタ
    Player();

    /// @brief プレイヤーを更新する
    /// @param stage ステージ情報
    void Update(const Stage& stage);

    /// @brief 現在の酸素量を取得する
    /// @return 酸素量
    double GetOxygen() const; // const correctness の徹底

private:
    /// @brief 酸素量(体力と同義)
    double oxygen_{ 100.0 }; // メンバ変数は末尾アンダースコア
};
```

5. プログラムの課題

本プロジェクトで採用した設計についての振り返りです.

- タイルセット画像の「1枚制限」:
- **現状**: Stage クラスの実装において，タイルセット画像は1枚に統合されていることを前提としています.
- **理由 (Pros)**: 描画バッチ処理を単純化し，開発初期のイテレーション速度を最大化するため（YAGNI原則の適用）.
- **改善案 (Consへの対策)**: 将来的にマップが大規模化し複数画像が必要になった場合は，`TileSet` クラスを分離し，GID（Global Grid ID）による画像管理テーブルを実装する予定です.

6. まとめ

本資料を根拠に，以下の能力をアピールいたします.

1. **設計力**: Strategyパターンによる「変更に強い」コード構築.
2. **実装力**: ツール連携・データ駆動による効率的な開発パイプラインの構築.
3. **品質意識**: 可読性の高いコーディングと，設計意図の説明能力.

ご清覧ありがとうございました.